More Advanced Graphics in R

Martyn Plummer

University of Warwick, UK

2 June 2026

Co-funded by
the European Union

Investing
in your future

Outline

Graphics Systems in R

R has several different graphics systems. Today we are going to talk about two of them:

- Base graphics (the `graphics` package)
- Grammar of graphics (the `ggplot2` package)

Base Graphics

- The oldest graphics system in R.
- Based on S graphics, as described in "The Blue Book"[1]
- Implemented in the base package `graphics`
  - Loaded automatically so always available
- Ink on paper model; once something is drawn "the ink is dry" and it cannot be erased or modified.

[1] Becker, Chambers and Wilks, *The New S Language, 1988*

Types of Plotting Functions

- High level
  - Create a new page of plots with reasonable default appearance.
- Low level
  - Draw elements of a plot on an existing page:
    - Draw title, subtitle, axes, legend . . .
    - Add points, lines, text, math expressions . . .
- Interactive
  - Querying mouse position (`locator`), highlighting points (`identify`)

The plot() function

The `plot()` function is a **generic** function.

- It does different things depending on what arguments you give it.
- The actual work is done by a **method** function. See `methods(plot)`.

Scatter plots

If the first argument to `plot()` is numeric, you get a scatter plot

- `plot(y)` Plot values of `y` against index (*i.e.* position in vector)
- `plot(x, y)` Plot values of `y` against corresponding values of `x`

Change the behaviour with the `type` argument

- `"l"` for lines
- `"p"` for points (the default)
- `"b"` for both
- plus quite a few more...

Statistical plots

barplot, hist	Bar plots and histograms
boxplot	Box plots
cdplot	Conditional density plots
dotchart	Dot plots
contour, image, persp	3-D visualisation
mosaicplot	Mosaic plots
pie	Pie charts

Scoping in base graphics (1/2)

Base plotting functions have the same scoping rules as any other R function

- Look for arguments in the calling environment ...
- Then the workspace...
- Then the rest of the search path ...

Scoping in base graphics (2/2)

What if my variables are in a data frame?

Use the `formula` method, which admits a `data` argument

```
plot(y ~ x, data=mydata)
```

Or wrap the plotting function in a call to `with()`

```
with(mydata, plot(x, y))
```

Or extract variables from the data frame

```
plot(mydata$x, mydata$y)
```

Customizing Plots in Base

- Most plotting functions take optional parameters to change the appearance of the plot
  - e.g., `xlab, ylab` to add informative axis labels
- Most of these parameters can be supplied to the `par()` function, which changes the default behaviour of subsequent plotting functions
- Look them up via `help(par)`! Here are some of the more commonly used:
  - Point and line characteristics: `pch, col, lty, lwd`
  - Multiframe layout: `mfrow, mfcol`
  - Axes: `xlim, ylim, xaxt, yaxt, log`

Adding to Plots in Base

- `title()` add a title above the plot
- `points()`, `lines()` adds points and (poly-)lines
- `text()` text strings at given coordinates
- `abline()` line given by coefficients (*a* and *b*) or by fitted linear model
- `axis()` adds an axis to one edge of the plot region. Allows some options not otherwise available.

Strategy for Customization of Base Graphics

- Start with default plots
- Modify parameters (using `par()` settings or plotting arguments)
- Add more graphics elements. Notice that there are graphics parameters that turn things *off*, e.g. `plot(x, y, xaxt="n")` so that you can add completely customized axes with the `axis` function.
- Put all your plotting commands in a script or inside a function so you can start again

Base Graphics Demo

```
library(ISwR)
par(mfrow=c(2,2))
matplot(intake)
matplot(t(intake))
matplot(t(intake),type="b")
matplot(t(intake),type="b",pch=1:11,col="black",
        lty="solid", xaxt="n")
axis(1,at=1:2,labels=names(intake))
```

Margins

- R sometimes seems to leave too much empty space around plots (especially in multi-frame layouts).
- There is a good reason for it: You might want to put something there (titles, axes).
- This is controlled by the `mar` parameter. By default, it is `c(5,4,4,2)+0.1`
  - The units are *lines of text*, so depend on the setting of `pointsize` and `cex`
  - The sides are indexed in clockwise order, starting at the bottom (1=bottom, 2=left, 3=top, 4=right)
- The `mtext` function is designed to write in the margins of the plot
- There is also an *outer margin* settable via the `oma` parameter. Useful for adding overall titles etc. to multiframe plots

Second Base Graphics Demo

```
x <- runif(50,0,2)
y <- runif(50,0,2)
plot(x, y, main="Main title", sub="subtitle",
     xlab="x-label", ylab="y-label")
text(0.6,0.6,"text at (0.6,0.6)")
abline(h=.6,v=.6)
for (side in 1:4)
   mtext(-1:4,side=side,at=.7,line=-1:4)
mtext(paste("side",1:4), side=1:4, line=-1,font=2)
```

Grammar of Graphics

- Originally described by Leland Wilkinson in the book *The Grammar of Graphics*, 1999 and implemented in the statistical software nViZn (part of SPSS)
- Statistical graphics, like natural languages, can be broken down into components that must be combined according to certain rules.
- Provides a *pattern language* for graphics:
  - geometries, statistics, scales, coordinate systems, aesthetics, themes, ...

Grammar of Graphics in R

- Implemented in R in the CRAN package `ggplot2`
- Described more fully by the `ggplot2` package author Hadley Wickham in the book *ggplot2: Elegant Graphics for Data Analysis*, 2009.
- Third edition (in progress) is available for free online at `https://ggplot2-book.org/`
- Built on top of the `grid` package

How ggplot works

- Start with a call to `ggplot` which defines
  - The data frame that contains all variables
  - The aesthetic mapping from variables to graphical parameters
- Add additional graphical components with `+`
  - Usually geometries (`geom_point`, `geom_line`, ...)
- GGplot will combine all graphical elements together

Scoping in ggplot

- Unlike base graphics, ggplot looks for variables in a data frame which must be supplied as an argument (or piped in)
- The data frame must be “tidy”
  1. Every column is a variable
  2. Every row is an observation
  3. Every cell is a single value
- Getting data into the right “tidy” format is often more work than plotting

GGplot Demo

```
ISwR::intake |>
  mutate(id=cur_group_rows()) |>
  pivot_longer(cols=c(pre,post)) |>
  mutate(name=factor(name, levels=c("pre", "post"))) ->
  intake

ggplot(intake,
       aes(x=name, y=value, group=id, shape=id)) +
  geom_point() +
  geom_line()
```

Graphics Devices

Graphics devices are used by all graphics systems.

- Plotting commands will draw on the current *graphics device*
- The *default* graphics device is a window on your screen:

  In RStudio RStudioGD()
  On Windows windows()
  On Unix/Linux x11()
  On Mac OS X quartz()

  It normally opens up automatically when you need it.
- You can have several graphics devices open at the same time (but only one is current)

Graphics Device in RStudio

RStudio has its own graphics device RStudioGD built into the graphical user interface

- You can see the contents in a temporary, larger window by clicking the zoom button.
- You can write the contents directly to a file with the export menu
- Sometimes the small size of the RStudioGD device causes problems. Open up a new device calling `RStudioGD()`. This will appear in its own window, free from the GUI.

Writing Graphs to Files

There are also non-interactive graphics devices that write to a file instead of the screen.

`pdf` produces Portable Document Format files

`win.metafile` produces Windows metafiles that can be included in Microsoft Office documents (windows only)

`postscript` produces postscript files

`png, bmp, jpeg` all produce bitmap graphics files

- Turn off a graphics device with `dev.off()`. Particularly important for non-interactive devices.
- Plots may look different in different devices